Programming in C

1.0 INTRODUCTION

1.1 Programming Language

A computer is directed by a series of instructions called **Computer Programs**, which specify a sequence of operations to be performed. Special languages are needed to write instructions, which can be understood by a computer. Natural languages such as **English** are not simple enough and cannot be interpreted by the computer. In early days of computer development, the codes used called **Machine Language**, were purely numerical, i.e. every instruction had to be written as a sequence of numbers and it was these numbers which the computer interpreted. Furthermore each computer had its own **machine code**. Later, more sophisticated or high-level languages, written in English letters, and using some simple mathematical notation, were developed. Examples of such high-level languages, include; FORTRAN, COBOL, BASIC, PASCAL, C, C++, and JAVA. Computers, however still only interprets their machine code, and special programmes called **compilers** have to be provided to translate the high-level language into this machine code.

1.2 C Programming

C is a general purpose programming language. Just like any other high-level language it is used for solving problems through a computer. Before a problem is put on a computer for solution, several tasks have to be carried out. These tasks may include but not limited to:

- Identifying the problem variables and their nature
- Specifying the objectives and scope of the problem and its solution requirements
- Constructing the necessary mathematical models (if needed) to depict the different problem as accurate as possible.
- Writing the program.
- Compiling the program

The program written in C or any other high-level language is called the **source program (or source code)**. The translated program in **machine code**, which is the one actually operated by the computer, is called the **object program (or object code)**. The process of translation is called **compiling the program** or compilation.

Important features of C include:

- Shorter expressions
- Modern control flow
- Modern data structures and
- Rich set of operators

C programming was originally designed for and implemented on the UNIX Operating System but it is not tied to any hardware or system. It can run on IBM machines, IBM compatibles and other systems. Compilers exist for different kinds of machines.

Definition 1

> **A computer program** is a set of instruction that tells a computer exactly what to do. The instruction, might tell the computer to add up a set of numbers, or compare two numbers and make a decision based on the result of comparison.

Definition 2

A programming language is an english-like language that you use to write your computer programs.

- There are many programming language, the most common ones are:
  - FORTRAN
  - COBOL
  - BASIC
  - Pascal
  - C
  - C++
  - Java

Definition 3

A compiler translates a computer program written in programming language into a form that a computer can execute.

- C is probably the most popular and widely used programming Language because it gives maximum control and efficiency to the programmer.
- Benefits you gain from learning C programming are:
  - Be able to read and write code for the largest number of platforms (i.e. Everything from micro controllers to nearly all modern operating systems)
  - The jump to the object oriented (C++) language becomes much easier. C++ is an extension of C.
  - Once you know C and C++ then Java can easily be learnt. Java is built upon C++.

1.3 Solving a Problem Using a Program

It is not always possible to solve real problem by closed form solution such as integral cos x-sin x. The problem solving is a multi-process as shown in figure 1 below.

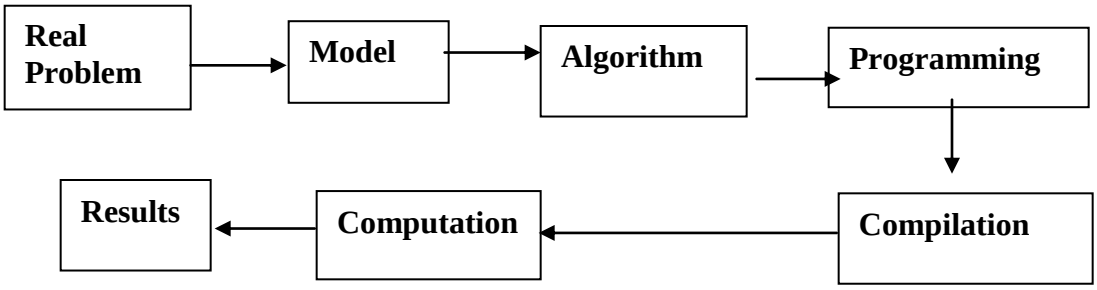


Figure 1: Steps of Problem Solving in a Computer

As seen in figure 1 above, a real problem is converted into a suitable mathematical model using simplified assumptions. A particular algorithm is used to solve the problem and the algorithm is programmed for a computer such that calculation may be performed. The computer gives result in the form of the **output**. If some data is needed before the computation is done, the process is called **data input.**

1.4 The General form of a C program

```
#include statements
function name()
           {
             Statement sequence
           }
```

The include statements are statements written at the top of your program to incorporate some standard library files which comes together with a C-compiler.

`function name()` this is a name of a function. All **C** programs consist of one or several functions.

`The opening brace` "{" : This marks the beginning of statement sequence.
`The close brace` "}" : This represents the formal conclusion of the program.

`Statement sequence`: May be one or several sequence of statements representing actions to be taken.

1.5 Starting with C

Let us consider the following program that prints out the line *"This is my first C- program"* save it as **myfirstP**

```
#include <stdio.h>
main()
     {
      printf("This is my first C - program");
     }
```

Compile the program then execute it.

If the program is wrongly typed, either it will not compile successfully or it will not run. If this happens edit the program and correct the errors and then compile again.

Program analysis

The statement `#include<stdio.h>` causes the file `stdio.h` to be read by the C compiler and be included in the program. It is one of the standard library files and contains information related to the `printf()` function.

The statement `main()` is the main function. All C programs must have a main function.

The statement `printf()` allows you to send output to the screen.

The character "**;**" is used as a statement terminator. It indicates the end of a statement. Every statement must be ended by a semicolon.

Exercise 1

Q1. Write a program to print your name. Save the file as MyName

Q2. Write a program to display First and Second names, Age, Gender and Address in the following format:

Name: Anderson James,
Age: 22yrs,
Address: P.O. Box 131,
City: MBEYA.

Hint: Use the code \n in your `printf()` function to indicate a request for a new line e.g. printf ("Name: Anderson James.\n");
Save the program as **Myaddress**

Q3. What are the major components of a C program?

1.6 Variables and Variable Types

Definition:

A variable is a named memory location that may contain some value.

A variable is an identifier that is used to represent some specified type of information within a designated portion of the program.

In C a variable represents a single data Item that can be a numeric quantity or a character constant. The data item must be assigned to a variable at some point in the program. The data item can be accessed later by referring to the variable name. A given variable can be assigned different data items at various places within the program. The information represented by the variable can change during the execution of the program. However, the data type associated to the variable cannot change.

Variable Declarations

A declaration associates a group of variables with specific data type. All variables must be declared before they can appear in executable statements.

A declaration consists of a data type followed by one or more variable names separated by a comma and ending with a semi-colon.

A C program may contain the following:

```
int a, b, c;
float root1, root2;
```

Thus, a, b and c are declared to be integer variables and root1 and root2 are floating-point variables.

In C, all variables must be declared before they can be used. A variable declaration tells the compiler what type of variable is being used. C supports 5 different basic data types:

DATA TYPE		KEYWORD	MIN RANGE
i	Character data	char	-127 to 127
ii	Signed whole numbers.	int	-32,767 to 32, 767
iii	Floating-point numbers (real number)	float	Six digits of precise
iv	Double precision floating-point number	double	Ten digits of precise
v	Valueless	void	-

Note:

- In C a variable declaration is treated as a statement and it must end with a semicolon (;). The programme may contain the following declarations:

  e.g. `int x;` Here the variable x is declared as an integer data type
  `int y;` Here the variable y is declared as an integer data type
  `float z;` Here the variable z is declared as a floating point value
  `char ch;` Here the variable ch is declared as a character data type
  `double d;` Here the variable d is declared as a double precision data type

- There are two places where variables are declared; inside a function, called **local variable** or outside the function, called **global variable**.
- A local variable is known to and may be accessed by only the function in which it is declared. Local variables cease to exist once the function that created them is completed. They are recreated each time a function is executed or called. Local variables are sometimes called **automatic variables.**
- A global variable can be accessed by any function in the program. These variables can be accessed (i.e. known) by any function comprising the program. They are implemented by associating memory locations with variable names. They do not get recreated if the function is recalled.
- C is case-sensitive; i.e. **x** and **X** are two completely different variables names
- To assign the variables x and y some values, we write,

```
x = 10;
y = 20;
```

- To assign the variable z and ch we write:

```
z = 5.6;
ch = 'N';
```

  Note that **`z`** being a floating point variable is assigned a real number (i.e. floating point value 5.6, and
  `ch` being a character variable data type is assigned a single character 'N' enclosed in single quotes.

- You can use `printf()` to display value of character integers and floating-point values. e.g. `printf("The answer is %d", 20);`

  where: %d is the format specifier for integer variable.
  %f is the format specifier for float variable.
  %c is the format specifier for character variable.

Example 1

This program declares a variable x as integer, assign it a value 10 and uses the `printf()` function to display the statement "The value of x is 10" (Save the program as **OneInt**)

```
#include <stdio.h>
main()
    {
          int x;
          x = 10;
         printf("The value of x is %d",x);
    }
```

Example 2

OneInt is modified to include one variable (Save the program as **MoreInt**)

The program declares a variables x and y as integers and assigns to them the values 10 and 2 respectively. It then uses the `printf()` function to display the statement
"The value of x is 10" and the statement "The value of y is 2"

```
#include<stdio.h>
main()
  {
     int x, y;
     x = 10;
     y = 2;
     printf("The value of x is %d",x);
     printf("The value of y is %d",y);
   }
```

Example 3

This program declares two different variables, count as integer, and y as float. The two variables are assigned some values and hence displayed on a screen.
(Save the program as **variable**)

```
#include <stdio.h>
main()
   {
     int count;
     float y;
     count = 12;
     y = 200.5;
     printf("This is DSTV\n");
     printf("with %d channels \n", count);
     printf("pay USD %f for installation", y);
  }
```

Exercise 2

Q1. Modify **MoreInt** and add another `printf()` so that it prints the value of x and y on the same line.

i.e. `printf("The value of x is %d and y is %d",x, y);`

Q2. Write a program that declares one integer variable called `num`. Give this variable the value 100 and then, using one `printf()` statement, display the value on the screen as follows:

100 is the value of num.

(Save the program as **num**)

Q3. Write a program to display your name, age, gender, address and region. Use variables for your age and gender. (Save the program as **mycv**)

1.7 Input Numbers from the Keyboard

There are several ways to input numeric values from the keyboard one of the easiest is to use another C's standard library functions called **`scanf()`**

e.g.
```
int num;
scanf("%d", &num);
```

where the "&" allows a function to place a value into one of its arguments.

Exercise 3

Q1. Write a program that will compute the area of a rectangle given its dimensions. Let the program first prompts the user for the length and width of the rectangle and then display the area. (Save the program as **rectangle_area**)

Q.2 Write a program that computes the area of a circle. Have the program prompt the user for each dimension (Save the program as **circle**).

Q.3 Write a program to compute the volume of a cylinder (Hint: use the mathematical model $\mathbf{v = \pi r^2 h}$). (Save the program as **cylinder**)

1.8 Comments

Definition:

A comment is a note to yourself (or others) that you put into your source code.

Note: All comments are ignored by the compiler.

- Comments are used primarily to document the meaning and purpose of your source code so that you can later remember the code.
- In C, comments starts by the symbol **/*** and ended by the symbol ***/**

  e.g. **/ * This is a C comment */**
- Comments can extend over several lines.

  e.g. /* This program computes
  the area of a rectangle */

2.0 CHARACTER SET

In C programming language, the character set refers to a set of all valid characters that can be used in the source program to form basic program elements such as constants, variables, operators and expressions. The C character set consists of uppercase letters **A** to **Z,** the lowercase letters **a** to **z,** the digits **0** to **9**, and certain special characters.

The following are special characters

!	*	+	\	“	<
#	(	=	|	{	>
‘	)	~	;	}	/
@	-	[	:	,	?
&	_	]	‘	.	(blank)

C uses combination of these characters such as \a, \b, \n, and \t to represent special conditions as shown below:

Combination of characters	**Meaning**
\a	alert/bell
\n	new line
\t	Horizontal tab

These character combinations are known as ESCAPE SEQUENCES

2.1 Identifiers

Identifies are names given to various program elements such as variables, function and arrays. Identifiers consist of letters and digits, in any order, except that the first character must be a letter. Lowercase and uppercase are permitted, though common usage favor is the use of lowercase letters. Upper and lower case letters are not inter changeable (i.e. an uppercase is not equivalent to the corresponding lowercase). An underscore (_) can also be included as it is considered to be a letter, and can take any position, even though commonly is used in the middle of an identifier. A character space is not used in identifier.

Examples:

The fallowing names are valid identifiers

x,	y1,	dog_1,	_temperature
name,	area,	tax_rate,	TABLE

The following names are not valid identifiers for the reasons stated.

1_day	the first character must be a letter
“x”	illegal character (“)
Order-no	illegal character (-)
tax rate	illegal character (blank space)

An identifier can be arbitrary long up to 31 characters. It is good practice, however, to use at most 8 characters in an identifier. As a rule identifies should contain enough characters so that its meaning is readily apparent. On the other hand excessive number of characters should be avoided.

2.2 Keywords

Keywords in C are reserved words that have standard pre-defined meaning. Keywords are used for their intended purpose only, and can not be used as programmer-defined identifiers.

Examples of the standard keywords are:

int	long	return	do	char	float
auto	break	case	const	continue	default
double	else	enum	extern	for	if
long	short	signed	static	switch	void
typedef	unsigned				

Note that the keywords are all lowercase.

2.3 Constants

The C programming language has four basic types of constants. These are:

i) integer constants
ii) floating point constants
iii) character constants
iv) string constants

Integer and floating point constants represent numbers. They are both referred to as **numeric-type constants.**

The following rules apply to numeric-type constants.

1. Commas and blank spaces cannot be included within numeric type constants.
2. The constants can be preceded by minus (-) sign, (an operator that changes the sign of positive constant)
3. The value of constants cannot exceed the specified minimum and maximum bounds.

2.3.1 Integer Constants

An integer constant is an integer-valued number; it consists of a sequence of digits and can be written in different number system. Decimal (base 10), Octal (base 8) and Hexadecimal (base 16). A decimal integer constant can consist of any combination of digits 0 through 9. If the constant contains two or more digits, the first digit must be something other than zero.

Valid decimal integer constants are shown below:

0 1 743 5280 32767 9999

Invalid decimal integer constants with reasons stated:

Invalid Integers	Reasons
12,245	illegal character (,)
56.0	illegal character (.)
10 20 30	illegal character (blank space)
123-45-6789	illegal character (-)
0900	the first charter cannot be zero.

Example 1
The quantity 3 x 10^5 can be represented in C by any of the following integer constants:

300000 3e5 3e +5 3E5 30E4 300e3

2.3.2 Floating Point Constants

A floating point constant is a base 10 number that contains either a decimal point or an exponent (or both).

Valid floating-point constants are shown below:

0.	1.	0.2	827.602
50000.	0.000743	12.3	315.0066
2E-8	0.006e-3	1.6667E+2	
.12121212e5			

Invalid floating-point constants are shown below with reasons stated.

Invalid Floating Point Constants	Reasons
1	either a decimal point or an exponent must be present
1,000.00	illegal character (,)
2E+10.2	the exponent must be an integer quantity (it cannot contain decimal point)
3E 10	illegal character (blank Space) in the exponent

The interpretation of a floating-point constant with exponent is essentially the same as for scientific notation except that the base 10 is replaced by E (or e). Thus

1.2 x 10^{-3} would be written as 1.2E-3
or 1.2e – 3
and this is equivalent to 0.12e – 2
or 12e – 4

Example 2
The quantity 5.026 x 10^{-17} can be represented in C by any of the following floating-point constants:

5.026E-17 .5026e-16 50.26e-18 0.0005026E –13

2.3.3 Character Constants

A character constant is a single character enclosed in apostrophes (single quotation marks)

Examples of character constants are:
'A', 'X', '3', 'A', ' '
Note that the last constant is a blank space enclosed in apostrophes.

Example 3
The following are character constants and their corresponding values as defined by the ASCII character set.

Constant	Value
'A'	65
'X'	120
'5'	53
'$'	36
' '	32

2.3.4 String Constants
A string constant is a sequence of characters enclosed in double quotes.

Examples of string constants are:
"MUST", "Amina", "Programming", " "

2.4 Escape Sequences

Escape sequence are characters that causes certain action to take place. In C these are usually a normal or special character preceded by a backslash (\).

The commonly used escape sequences are:

Character	Escape Sequence	ASCII Value
New line	\n (line feed)	010
Horizontal tab	\t	009
Bell alert	\a	007
Backspace	\b	008
Vertical tabs	\v	011
Form feed	\f	012
Carriage return	\r	013
Quotation mark	(\")	034
Question mark	(\?)	063
Back slash	(\")	092
Null	\0	000

2.5 ASCII Character Set

All personal computers make use of the American Standard Code for Information Interchange (ASCII) character set, in which each individual character is numerically encoded with its own unique 7-bit combination (Hence a total of 2^7 =128 different characters. The table 1 below contains the ASCII character set.

Table 1: The ASCII Character Set

ASCII Value	Character	ASCII Value	Character	ASCII Value	Character	ASCII Value	Character
000	NUL	032	blank	064	@	096	`
001	SOH	033	!	065	A	097	a
002	STX	034	“	066	B	098	b
003	ETX	035	#	067	C	099	c
004	EOT	036	$	068	D	100	d
005	ENO	037	%	069	E	101	e
006	ACK	038	&	070	F	102	f
007	BEL	039	‘	071	G	103	g
008	BS	040	(	072	H	104	h
009	HT	041	)	073	I	105	i
010	LF	042	*	074	J	106	j
011	VT	043	+	075	K	107	k
012	FF	044	‘	076	L	108	l
013	CR	045	-	077	M	119	m
014	SO	046	.	078	N	110	n
015	SI	047	/	079	O	111	o
016	DLE	048	0	080	P	112	p
017	DCI	049	1	081	Q	113	q
018	DC2	050	2	082	R	114	r
019	DC3	051	3	083	S	115	s
020	DC4	052	4	084	T	116	t
021	NAK	053	5	085	U	117	u
022	SYN	054	6	086	V	118	v
023	ETB	055	7	087	W	119	w
024	CAN	056	8	088	X	120	x
025	EM	057	9	089	Y	121	y
026	SUB	058	:	090	Z	122	z
027	ESC	059	;	091	[	123	{
028	FS	060	<	092	\	124	|
02	GS	061	=	093	]	125	}
030	RS	062	>	094	↑	126	~
031	US	063	?	095	-	127	DEL

Note: The first 32 characters and the last character are control characters; they cannot be printed.

3.0 OPERATORS

An operator is a symbol specifying an operation to be performed. The C language contains a rich set of operators such as:-

(i) Arithmetic Operators
(ii) Relational Operators
(iii) Logical Operators
(iv) Assignment Operators

3.1.1 Arithmetic Operators

Arithmetic operators are those operators that perform arithmetic (numeric) operations. The following table shows all the arithmetic operators supported by the C language.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Note: The modulo operator (%) can be used with integer types only
The modulus operator produces the remainder of an integer division

```
e.g.  5  % 2 = 1
      4 % 2 = 0
```

3.1.2 Relational Operators

The relational operators compare two values and return a true or false based upon the comparison of the result. The relational operators include the following:

Operator	Meaning
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
= =	Equal
! =	Not equal

3.1.3 Logical Operators

The logical operator is a symbol or word used to connect two or more expressions such that the result produced depends only on that of the original expressions and on the meaning of the operator. The logical operators basically connect together True/False results. Common logical operators include:-

Operator	Action
&&	AND
||	OR
!	NOT

Practical examples on usage of relational and logical operators are covered later.

3.1.4 Assignment Operators

Assignment operators are the operators used to assign values to variables. The following table lists the assignment operators supported by the C language.

Operator	Description
=	**Simple assignment** operator. Assigns values from right side operand to left side operand.
+ =	**Add** AND **assignment** operator. It adds the right operand to the left operand and assigns the result to the left operand.
- =	**Subtract** AND **assignment** operator. It subtracts the right operand from the left operand and assigns the result to the left operand.
* =	**Multiply** AND **assignment** operator**.** It multiplies the right operand with the left operand and assigns the result to the left operand
/ =	**Divide** AND **assignment** operator. It divides the left operand with the right operand and assigns the result to the left operand.
% =	**Modulus** AND **assignment** operator. It takes modulus using two operands and assigns the result to the left operand

The following assignment operators have the following meanings

```
a  +=  b   ⟺   a  = a + b
x  -=  y   ⟺   x  = x - y
a  /=  b   ⟺   a  = a / b
a  *=  b   ⟺   a  = a * b
a  %   b   ⟺   a  = a % b
```

3.2 Order of Precedence

Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated. Certain operators have higher precedence than others. It is necessary to be careful of the meaning of each expression.

Example 1

```
a + b * c
```

We may want the effect as either

```
(a + b) * C
     or
a + (b * c)
```

All operators have a priority. High priority operators are evaluated before lower priority ones. Operators of the same priority are evaluated from left to right, so that

```
a - b - c
```

is evaluated as

```
( a - b ) - c
```
as you would expect.

The order for all C operators is as given in the table below:

Operator Category	Operators	Associativity
Unary operator	- ++ -- ! size of *(type)*	R → L
Arithmetic multiply, divide and reminder	* / %	L → R
Arithmetic add and subtract	+ -	L → R
Relational operators	< <= > >=	L → R
Equality operators	= = !=	L → R
Logical AND	&&	L → R
Logical OR	||	L → R
Assignment operator	= += - = * = / = % =	R → L

Example 2

Suppose that x, y and z are variables which have been assigned the value 2, 3, and 4 respectively. Then the expression

```
x *= -2 * (y + z) / 3
```

Is equivalent to the expression

```
x = (x * ( -2 * (y + z) / 3))
```

The expression will cause the value –9.33 to be assigned to x.

Furthermore, the expression:

```
x < 10 && 2 * y < z
```

is interpreted as

```
(x < 10) && ( ( 2 * y ) < z ))
```

And will be evaluated to False, F.

Example 3

Suppose a, b, and c are integer variables that have been assigned the values a = 8, b = 3 and c = -5. Determine the value of each of the following.

```
(a)  a + b + c     (b)  2 * b + 3 + (a - c)     (c) a / b
(d)  a % b         (e) a * (c % b)              (f)(a * c) % b
(g) a * b / c
```

3.3 Expressions and Statements

3.3.1 Expressions

An expression represents a single data item, such as number or a character. The expression may consist of a single entity, such as a constant, a variable an array element or a reference to a function. An expression may also consist of a combination of such entities connected by one or more operators.

Expression can also represent logical conditions that are either **TRUE** or **FALSE**. In C the conditions TRUE and FALSE are represented by the integer values 1 and 0, respectively.

Examples of expression

a + b

x = y

c = a + b

c = = y

x <= y

++i

The first expression involves the use of additional operator (+)

The second expression involves the assignment operator (=)

The third expression combines the features of the first two expressions. In this case the value of the expression **a + b** is assigned to the variable **c**

The fourth expression is the test of equality. Thus, the expression will have the valve 1 (TRUE) if the value of **x** is equal to the value of **y**. Otherwise, the expression will have the value 0 (FALSE).

The last expression causes the value of the variable 1 to be increased by 1. The expression.

++i

is thus equivalent to

i = 1+ i

The operator **++i** is called the Unary operator.

3.3.2 Statements

A statement causes a computer to carry some action. There are three different types of statements in C.

i. Expression statement
ii. Compound statement
iii. Control statement

An **expression statement** consists of an expression followed by a semi-colon. The execution of an expression statement causes the expression to be evaluated.

Example of expression statement

```
a  =  3;
c  =  a  +  b;
++i;
printf("Area = %f", area);
;
```

The statement consisting of a semicolon is called an empty or NULL statement.

A **compound statement** consists of several individual statements enclosed within a pair of braces ({ }). The individual statements may themselves be **expressions statements**, **compound statements** or **control statements**. The compound statements provide capability for embedding statements within other statements. Unlike expression statements, a compound statement does not end within a semi-colon.

Examples of compound statement

```
{
   pi = 3.141593;
   circumference = 2 * pi * radius;
   area = pi * radius * radius;
}
```

This compound statement consists of three assignment-type expression statement, though it is considered a single entity within the program in which it appears.

Note that compound statement does not end with a semicolon after the brace.

Control statements are used to create special program features such as logical tests, loops and branches. Many control statement require that other statements embedded within them as shown in the example below:

Example

```
while (count < = n)
     {
        printf("x = ");
        scanf("%f", & x);
        sum += x;
        ++count;
     }
```

This statement consists of a compound statement, which in turn contains four expressions statements. The compound statement will continue to execute as long as long as the value of count does not exceed the value of n.

3.4 Symbolic Constants

A symbolic constant is a name given to some numeric constant, or a character constant or string constant, or any other constants. **Symbolic constant names** are also known as constant identifiers. Pre-processor directive `#define` is used for defining symbolic constants.

Syntax for Creating Symbolic Constants

`#define` **symbolic_constant_name value_of_the_constant**

Examples of Symbolic Constants

`#define` PI 3.141592
`#define` GOLDENRATIO 1.6
`#define` MAX 500

Exercise 4

Q.1 Summarizes the rules for naming identifiers. Are upper case letters equivalent to lowercase letters?

Q.2 Name and describe the four basic data types in C?

Q.3 Describe two different ways that floating-point constants can be described?

Q.4 What is an expression? What are its components?

Q. 5 A C Program contains the following expressions:

```
int     i = 8,    j =5;
float   x = 0.005,      y =  -0.01;
char    c = 'c',        d = 'd';
```

Determine the value of each of the following expressions. Use the value initially assigned to the variables for each expression.

a) x > = 0

b) (3 * i – 2 * j) % (2 * d – c)

c) 2 * (i/5) + (4 * (j – 3)) % (i + j – 2)

d) – (i + j)

e) ++ i

f) --j

g) i < = j

h) c > d

i) c = = 99

j) (2 * x + y) = = 0

k) 5 * (i + j) > ‘c’

l) (2 * x + (y = = 0)

m) (i > 0) && (j < 5)

n) (i > 0) || (j < 5)

Q. 7 Given

x = 1, y = 4, z = 2

Determine the following.

x + y mod z * 5 – y

Q. 8 Given x = 4; b = 6, c = 8

Deter Determine the value of the following:

(a +b * c) < (4 * 6+ 8)

Q. 9. Given x = TRUE, y = TRUE, z = FALSE

Find the value of the following:

(i) x && y

(ii) (x && y) && z

(iii) x && (y && z)

(iv) x < y || y > z

(v) (x || y) || z

Q. 10 A C program contains the following declarations**:**

int i = 8, j =5, k;

float x = 0.005, y = -0.01, z;

char a, b, c = ‘c’, d = ‘d’;

Determine the value of each of the following:

(a) k = (i + j) (b) z = k = x (c) i = j

(d) k = (x + y) (e) i = j = 1.1 (f) k = c

(g) z = i / j (h) i += 2 (i) i /= j

(j) i %= j (k) y - = x (l) i += (j - 2)

4.0 PROGRAM CONTROL STATEMENTS

Control statements are statements which control the flow of execution of the program.

Types of Control Statements in C

- If statements
- Switch Statement
- Loop Statements

4.1 The "*if* … and *if* … *else*" Statements

The `if` statement is one of C's selection or conditional statement. Its operation is governed by the outcome of a conditional test that evaluate to either true or false. Generally, the syntax of the `if` statement is:

```
if( expression)
   statement;
```

where `expression` is any expression and `statement` is any statement.

Commonly the expression inside the *if* compares one value with another using a relational operator, such as >, < and = = (i.e. greater than, less than and equal operators respectively)

Example 1

```
if (10 > 9)
    printf("This is true");
```

This statement will cause the message; **"**`This is true`" to be displayed when the expression evaluates true.

Example 2

```
if(5 > 9)
   printf("This will not print");
```

In this statement "`This will not print`" will not be displayed, because the condition is false. The statement is therefore bypassed.

If you have a series of statements, no matter all of which should be executed together or not depending on whether some condition is true, in this case you enclose them in braces:

```
if( expression)
     {
       Statement1;
       Statement2;
       Statement3;
           .
           .
           .
       Statementn;
     }
```

Example 3

This program prompts the user to enter an integer number and uses *if* conditional test to evaluate the number, (if positive or negative. The program then prints the result of valuation. Type, compile and execute the program. (Save the program as numsign)

```
#include <stdio.h>
main()
  {
     int num;
     printf("Enter an integer:   ");
     scanf("%d", &num);

     if (num < 0)
        printf("The number is negative.");

     if (num > -1)
        printf("The number is non-negative.");
  }
```

Example 4

This program converts meters to kilometers or kilometers to meters depending upon what the user requests. (Type the program and save it as metres)

```
#include<stdio.h>
main ()
  {
      float number, ans, sol;
      int choice;
      printf("Enter a number for convention:   ");
      scanf("%f",&number);

      printf("MENU:\n\n");

      printf("1: meters to kilometers.\n");
      printf("2: kilometers to meters\n\n");

      printf("Enter your choice: ");
      scanf("%d", &choice);

      if (choice == 1)
          {
            ans = number/1000;
            printf ("%f km", ans);
          }
      if (choice == 2)
          {
            sol = number * 1000;
            printf ("%f m", sol);
          }
   }
```

The addition of optionally `else` statement provides a two-way decision path. The "`else`" clause, is to be executed if the condition is not met. The statement looks like this

```
if (expression)
     Statement 1;
else
     Statement 2;
```

If the `expression` is true, then statement 1 or block of statements will execute and the `else` portion is skipped. However, if the `expression` is false, then statements 2 or block of statements (following the keyword `else`) will execute and statement 1 is bypassed.

Example 5

```
if(n > 0)
     average = sum / n;
else
   {
     printf("Can't compute the average\n");
     average = 0;
   }
```

Example 6

Modify **numsign.c** program and delete the second `if` statement and write the program as follows:

```
#include <stdio.h>
main( )
  {
     int num;
     printf("Enter an integer:   ");
     scanf("%d", &num);

     if (num < 0)
          printf("The number is negative.");

     else
          printf("The number is non-negative.");
   }
```

Example 7

This program prompts the user for two numbers, divides the first by the second and displays the result. However, division by zero is not allowed (in common language we say it is **undefined,** so the program uses an `if` and an `else` statement to prevent division by zero from occurring (Type the following program and save it as **Else**).

```
#include <stdio.h>
main( )
   {
     int num1,num2;
     printf("enter first number: ");
     scanf("%d",&num1);
     printf("Enter a second number: ");
```

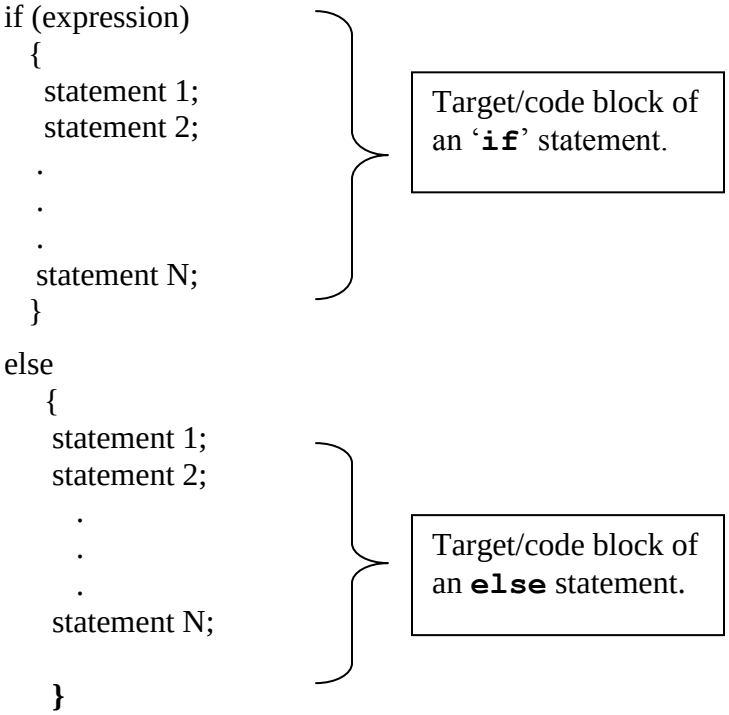
```
scanf("%d",&num2);

if (num2 == 0)
     printf("Cannot divide by zero\n");

else
     printf("The Answer is: %d", num1/num2);
}
```

Note: The target of an **`if`** statement need not only be a single statement. In C you can link two or more statements together. This is called a **block of code** or a **code block** and they are normally surrounded with the opening and closing curly braces.

Example



Example 8

This program is an improved version of **metres.c** program. Notice the use of code blocks.

```
#include<stdio.h>
main ( )
{
float num, ans, sol;
int choice;
printf("MENU:\n\n");
printf("1: meters to kilometers.\n");
printf("2: kilometers to meters. \n");
printf("Enter choice:   ");
scanf("%d", &choice);

if (choice == 1)
  {
```

```
    printf("Enter number of meters:    ");
    scanf("%f", &num);
    ans = num/1000;
    printf("%f km",ans);
  }
else
  {
    printf("Enter number of  kilometres:     ");
    scanf("%f", &num);
    sol = num * 1000;
    printf(" %f m", sol);
  }
}
```

4.2 Nested `if` Statements:

It is possible to string together several *`if`* and *`else`* into what is sometimes called an *if-else-if* ladder or *if-else-if staircase*, or *nested if statements*.

Its general form is:

```
if (expression1)
      statement1;
else if (expression2)
      statement2;
else if (expression3)
      statement3;
else
      statement4;
```

You can nest "if" at most 15 levels deep

Example 9

Open the **operation.c** and replace the last 3 if statement with:

```
  if (ch=='A')
     printf("%d", a + b);
else if (ch == 'S')
     printf("%d", a - b);
else if (ch == 'M')
     printf("%d", a * b);
else if (ch == 'D' and b != 0)
     printf("%d", a/b);
```

It's also possible to nest one `if` statement inside another. (For that matter, it's in general possible to nest any kind of statement or control flow construct within another.) For example, here is a little piece of code which decides roughly which quadrant of the compass you're walking into, based on an x value which is positive if you're walking east, and a y value which

is positive if you're walking north:

```
if(x > 0)
  {
      if(y > 0)
         printf("Northeast.\n");
      else
         printf("Southeast.\n");
   }
      if(y < 0)
            printf("Northwest.\n");

      else  printf("Southwest.\n");
   }
```

When you have one an `if` statement (or loop) nested inside another, it's a very good idea to use explicit braces { }, as shown, to make it clear (both to you and to the compiler) how they're nested and which `else` goes with which `if`. It's also a good idea to indent the various levels, also as shown, to make the code more readable to humans. Why do both? You use indentation to make the code visually more readable to yourself and other humans, but the compiler doesn't pay attention to the indentation (since all white-space is essentially equivalent and is essentially ignored). Therefore, you also have to make sure that the punctuation is right.

Here is an example of another common arrangement of `if` and else. Suppose we have a variable grade containing a student's numeric grade, and we want to print out the corresponding letter grade. Here is code that would do the job:

```
      if(marks >= 70)
            printf("A");
      else if(marks >= 60)
            printf("B+");
      else if(marks >= 50)
            printf("B");
      else if(marks >= 40)
            printf("C");
      else if(marks >= 35)
            printf("D");
      else  printf("F");
```

What happens here is that exactly one of the six `printf()` calls is executed, depending on which of the conditions is true. Each condition is tested in turn, and if one is true, the corresponding statement is executed, and the rest are skipped. If none of the conditions is true, we fall through to the last one, printing ``F''.

In the cascaded `if/else/if/else/...` chain, each else clause is another `if` statement. This may be more obvious at first if we reformat the example, including every set of braces and indenting each if statement relative to the previous one:

```
if(grade >= 70)
              {
              printf("A");
              }
       else   {
              if(grade >= 60)
                     {
                     printf("B+");
                     }
              else   {
                     if(grade >= 60)
                            {
                            printf("B");
                            }
                     else   {
                            if(grade >= 40)
                                   {
                                   printf("C");
                                   }
                     else   {
                            if(grade >= 35)
                                   {
                                   printf("D");
                                   }

                            else   {
                                   printf("F");
                                   }
                            }
                     }
              }
```

By examining the code this way, it should be obvious that exactly one of the printf calls is executed, and that whenever one of the conditions is found true, the remaining conditions do not need to be checked and none of the later statements within the chain will be executed. But once you've convinced yourself of this and learned to recognize the idiom, it's generally preferable to arrange the statements as in the first example, without trying to indent each successive if statement one tabstop further out.

4.3 Reading Characters from the Keyboard using `getchar()` and `getche()` Function

- C defines a function called **`getchar(),`** which returns a single character typed on the keyboard. When called, the function waits for a key to be pressed. After the character is typed you press the Enter key.
- Another one is the **`getche()`** function which is similar to **`getchar()`** except that it does not require to press the Enter key. This function requires a header file called **`conio.h`**

Example

The following program uses a character input to obtain a menu selection, it then allows the user to add, subtract, multiply and divide two numbers. (Save the program as **operation.c**)

```
#include <stdio.h>
main( )
   {
     int a, b;
     char ch;

     printf("Do you want to:\n\n");
     printf("Add, Subtract, Multiply, or Divide?\n\n");

     printf("Enter your choice:  A for Add,  S for Subtract, M for Multiply, and D for Divide:    ");
     ch = getchar( );
     printf("Enter first no:  ");
     scanf("%d", &a);
     printf("Enter second number:   ");
     scanf("%d", &b);

     if (ch == 'A')
          printf("%d", a + b);
     if (ch == 'S')
          printf("%d", a - b);
     if (ch == 'M')
          printf("%d", a * b);
     if (ch == 'D'&& b!= 0)
          printf("%d", a/b);
   }
```

Exercise

Replace the `getchar()` in **Operation** with the `getche()` and observe the differences.

4.4 The `Switch` Statement

While "**`if`**" is good for choosing between two alternatives, it quickly becomes cumbersome when several alternatives are needed. Solution to this problem is the '**`switch`**' statement. The **`switch`** statement in C tests the value of a variable and compares it with multiple cases. Once the case match is found, a block of statements associated with that particular case is executed. If a case match is NOT found, then the default statement is executed, and the control goes out of the switch block.

The general form of a `switch` statement is:

```
switch(value)
     {
     case  constant1:
           statement sequence;
           break;
     case  constant2:
           statement sequence;
           break;
     default:
           Statement sequence;
           break;
     }
```

How it works:

- Each case is labeled by one or more integer-valued constant expression. A value is successively tested against a list of integer or character constants.
- When a match is found, the statement sequence associated with that match is executed starting from that case. Execution continues until **`break`** is encountered.
- The default statement sequence is performed if no matches are found. The 'default' is optional. If all matches fail and default is absent, no action takes place.
- Cases and default clauses can occur in any order.

Example 1

This program recognizes the numbers 1, 2, 3 and 4 and prints the name of the number you enter. (Save the file as **switch.c**)

```
#include<stdio.h>
main()
  {
     int I;
     printf("Enter a number between 1 & 4 inclusive:   ");
     scanf("%d", &I);
     switch (I)
        {
          case 1:
               printf("The number you entered is One");
               break;

          case 2:
               printf("The number you entered is Two");
               break;

          case 3:
               printf("The number you entered is Three");
               break;
```

```
            case 4:
                 printf(“The number you entered is Four”);
                 break;

            default:
                 printf(“Unrecogniezed Input!”);
        }
    }
```

Example 2

This program prints days of the week. (Save the file as **days.c**)

```
#include <stdio.h>
main()
{
    int day;
    printf("Enter day number(1-7): ");
    scanf("%d", &day);

    switch(day)
       {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        case 4:
            printf("Thursday");
            break;
        case 5:
            printf("Friday");
            break;
        case 6:
            printf("Saturday");
            break;
        case 7:
            printf("Sunday");
            break;
        default:
            printf("Invalid input! Please enter day number between 1-7.");
    }

 }
```

Differences Between "`if`" and "`switch`" Statements

'if' statement	"switch" statements
i) "**if**" conditional expression can be tested using relational or logical operators. ii) "**if**" works for any data type	i) "**switch**" can only test for equality. ii) "**switch**" will work with only "**int**" or char types: you can't for example use floating-point numbers.

Exercise

(i) Write a C program to find maximum between two numbers using switch case.
(ii) Write a C program to check whether a number is even or odd using switch case.
(iii) Write a C program to check whether a number is positive, negative or zero using switch case.
(iv) Write a C program to find roots of a quadratic equation using switch case.
(v) Write a C program to create Simple Calculator using switch case.

Loop

Loop is used to execute the block of code repeatedly for a specified number of times or until it meets a specified condition. In C Programming Language we have following types of loop.

(i) `for` loop
(ii) `while` loop
(iii) `do while` loop

4.4 The `for` Loop

The `for` loop is one of C's three loop statements. The `for` loop is used to repeat a statement or block of statements a specified number of times.

The general form of the for loop is

```
for (initialization; conditional_test; increment)
     statement;
```

Where the `initialization` is used to give an initial value to the loop-control variable.
The `conditional_test` is used to test the loop control variable against a target value. If the conditional test evaluates true, the loop repeats, if it is false, the loop stops and the program execution picks up with the next line of code that follow the loop.
The **increment** is used to increase (or decrease) the loop-control value by a certain amount. The expression for the increment is as follows:

```
for (i = initial_i; i <= i_max; i = i + i_increment)
          {
              block of statements;
          }
```

The following program code prints the first five natural numbers

/* Print the first five Natural numbers */

```
for(num = 1; num <= 5; num++)
     printf("%d\n", num);
```

The expression `num++` is called the unary operator, it is the same as

```
num = num + 1
```

Example 1

The following program uses a for loop to print the numbers 1 through 10 on the screen (Type the program and save it by the name **tennum**)

```
#include <stdio.h>
main()
  {
     int num;
     for(num = 1; num < 11; num = num + 1)
          printf("%d ", num);
          printf("The End. ");
  }
```

This program shall produce the following output:

1 2 3 4 5 6 7 8 9 10 The End.

Program analysis:

The program works like this:

- First, the loop control variable `num` is initialized to 1
- Second, the Boolean expression `num < 11` is evaluated; as long as it is true, the for loop begins running. After the number is printed, `num` is incremented by 1 and the conditional test is evaluated again.
- Third, this process continues until num = 11. When this happens the `for` loop stops and the programme execution picks up next statement `printf("The end")`

Example 2

This program computes the product and sum of the numbers from 1 to 5 (Type the program and save it as **sum,** then compile and run the program)

```
#include <stdio.h>
main( )
  {
     int num, sum, prod;
     sum = 0;
     prod = 1;
     for (num =1; num < 6; num++)   /* num ++ is the same as */
          {                         /*     num = num + 1 */
            sum = sum + num;
            prod = prod * num;
          }
```

```
    printf("product and sum are: %d, %d", prod, sum);

}
```

A `for` loop can also run negatively

e.g. `for(num = 20; num > 0; num = num - 1) /* num-- */`

Exercise 5

Q1. Write a program that asks for the sums of the numbers between 1 and 10. That is, it asks for 1 + 1 then 2 + 2 and so on. (Save the program as **newsum**)

Q2. Write a program that prompts the user for an integer value. Next, using a `for` loop make it count down from this value to zero displaying each number on its own line. When it reaches zero have it sound the bell. (save the program as **countd**) (Hint: for a bell to sound use the following printf ("\a")

Q3. Create a program that prints the numbers from 17 to 60. (save the program as **17_ to_ 60**)

Q4. Write a program that prints the numbers between 30 and 100 that can be evenly divided by 3 (save the program as **divby3**)

Q5. Write a program that prints even numbers between 10 and 99 (save the program as **even**)

Q6. Write a program that prints the numbers 1 to 100 using 5 columns. Have each number separated from the next by a tab. (Save the program as **5col**)

More examples of the `for` loop

The next segment calculates and prints even numbers using related natural numbers in the counter. Note that since more than one statement needs to be repeated, they are enclosed by curly brackets. If not bounded, only the first statement is repeated. Other statement after the first one are executed when the program exit the loop.

```
/* Print the first five even numbers */
for(i = 1; i <= 5; i++)
    {
      num = i * 2;
      printf("%d\n", num);
    }
```

Alternatively the same numbers can be printed by the following code:

```
for (num = 2; num <= 10; num = num + 2)
   printf("%d\n", num);
```

It is also possible to use a negative increment (i.e., decrement) using the "--" operator or an assignment statement.

/* Print the first five odd Natural numbers */

```
for(i = 5; i >= 1; i--)    /* the operator "- - " is equivalent to i = i –1 */
  {                        /* it decrements the counter i  by 1*/
```

```
    num = i * 2 - 1;
    printf("%d\n", num);
  }

for(num = 9; num >= 1; num = num - 2)
   printf("%d\n", num);
```

4.5 Counted Repetition with the for Loop (Loop Iteration)

When the same operation is to be done over and over again for a specified number of times the **`for`** loop is usually the preferred programming structure. In this case a **`for`** loop contains the statements that are to be repeated. These statements are enclosed by curly brackets, forming what is called **a compound statement.** A variable is used to keep track of which iteration of the loop is being done. This is known as the "**loop control variable**" and is given the name **counter** in this program, since we are having the computer counting the iterations.

In the next example the program computes and prints the area of a triangle three times given dimensions. i.e. the same operations are performed three times, we will use the natural numbers from one (1) to three (3).

```
#include<stdio.h>   /* standard I/O header file */
main()
{
  int b, h;           /* base, height */
  float A;            /* Area */
  int counter;        /* Loop control variable */

  printf("This program calculates and prints the areas of three triangles\n");
  printf("one after another after you enter their dimensions.\n");
  printf("When asked to type in a dimension and hit the ENTER key.\n");
  printf("\n");
  for(counter = 1; counter <= 3; counter++)
       {
        /*    Inputting Data    */
          printf("Triangle #%d:\n", counter);
          printf("What is the length of the triangle's base? ");
          scanf("%d", &b);
          printf("What is the triangle's height? ");
          scanf("%d", &h);

        /* Calculation */
           A = b * h / 2.0;
        /* Printing Results */
          printf("\n");
          printf("The area of a triangle with a base of %d units and\n", b);
          printf("a height of %d units is %.1f square units.\n", h, A);
       }              /* End of for loop */
}
```

4.6 The `while` Loop

Another C loop statement is a **`while`** loop. It has this general form:

```
initialization;
while (condition)
      {
      statement; (or block of statements)
      increment;
      }
```

The *`while`* loop works by repeating its target as long as the expression is true. When it becomes false, the loop stops. The value of the expression is checked at the top of the loop. If the expression is false to begin with, the loop will not execute even once.
The *`while`* loop continues to loop until the conditional expression becomes false. The condition is tested upon entering the loop. Any logical construction can be used in this context.

A typical while loop may have the following statements:

```
i = initial_i;

   while(i <= i_max)
       {
     ...block of statements...

     i = i + i_increment;
       }
```

while loop

To a good extent the `for` loop discussed above can be used instead of `while` loop and still produce the same results with fewer statements.

For instance the while loop structure given above may be re-written in the easier syntax of the `for` loop as follows:

```
for(i = initial_i; i <= i_max; i = i + i_increment)
    {
      block of statements;
    }
```

for loop with fewer statements

Infinite loops are possible e.g. `for(;;),` but not too good for your computer budget! C permits you to write an infinite loop, and provides the **`break`** statement to ``breakout " of the loop. For example, consider the following (admittedly not-so-clean) re-write of the previous loop:

```
angle_degree = 0;
       while(angle_degree >= 0)
           {
             printf("%d\n", angle_degree);
             angle_deree = angle_degree  + 60;
           }
```

It can be noted easily that this code will execute indefinitely, forming what is called an **infinite loop**. Such a loop can be broken down by adding the "breakout" statement as shown below.

```
angle_degree = 0;
 while (angle_degree >= 0)
    {
      printf("%d\n", angle_degree);
      angle_degree = angle_degree + 30;
      if (angle_degree == 360)
      break;
    }
```

The conditional **`if`** simply asks whether angle_degree is equal to 360 or not; if yes, the loop is stopped.

Example 1

The following program uses a `while` loop to print the numbers from 1 to 4 on the screen (Type the program and save a **fournum**)

```
#include <stdio.h>
main()
    {
   int count=1;
   while (count <= 4)
        {
        printf("%d ", count);
        count++;
        }
    }
```

Example 2

What do you think will be the output of this program?

```
#include <stdio.h>
main()
     {
     int var=1;
     while (var <=2)
        {
        printf("%d ", var);
        }
     }
```

4.7 The *do while* Loop

Contrary to the **`while`** loop which test the termination condition at the top, the **`do while`** loop has the feature that the exit-condition check is built into the end of the loop. That is, it tests the condition at the bottom after making each pass through the loop body; and the body is always executed at least once.

The syntax of the **`do while`** loop is:

```
Initialization;
 do   {
       statement(s);
       increment;
      }while (condition);
```

The `statement` is executed, then `condition` is executed. If it is it is evaluated true the statement is executed again, and so on. When expression becomes false, the loop terminates.

The following code is an example of a **`do while`** loop and prints the first five natural numbers

```
/* Print the first five Natural numbers */
        num = 1;
        do {
             printf("%d\n", num);
             num = num + 1;
           } while (num <= 5);
```

Before the loop begins, the counter called **`num`** is initialized to the first value desired. The loop itself consists of four lines:

(1) The reserved word **`do`** to indicate the beginning,
(2) A statement to print the value of n**`um`**,
(3) An assignment statement to increment the value of **`num`** by one, and
(4) The reserved word **`while`** with a condition inside parentheses to indicate the end.

After each iteration of the statements inside the loop, the condition is checked. If the condition is false, the loop is exited.

Note: The statements inside the loop will always be executed at least once, because the condition is not checked until after the statements have been executed.

Example 1

This program prints the numbers 1 to 5 using a **`do while`** loop. Save the program as **1to5.c**

```
#include<stdio.h>
main()
        {
           int I;
           I = 0;
        do
            {
              printf("The value of I is now %d\n", I);
              I  =  I + 1;
            } while( I < 6);
        }
```

Example 2

The **do** loop is especially useful when your program is waiting for some event to occur. This program waits for the user to type a letter q. (Save the program as **doq.c**)

```
#include<stdio.h>
#include<conio.h>
main()
  {
     char ch;
     do {
          ch = getche();
        } while (ch != 'q');
     printf("Found a q");
   }
```

Exercise

Q1. Write a program that converts miles to kilometers using a "`do while`" loop, allow the user to repeat the conversion (one mile = 1.6093). Save the program as **mile.c**

Q2. Write a program that displays the menu below and uses the "`do while`" loop to check for valid responses. Your program does not need to implement the actual function shown in the menu. Save the program as **menu.c**

More examples on the "`do while`" loop

Example 3

Let us modify our **degree.c** so that it prints the conversion automatically (save the program as **newdge.c**)

```
#include <stdio.h>
main( )
  {
     float C,F;
     C = 0;
     while (C <= 100)
          {
          for (C = 0; C <= 100;  C += 10)
            {
               F =  9.0/5.0 * C + 32;
               printf ("%f degree C  = %f degrees F\n", C, F);
            }
          }
    }
```

Example 4

This program continues to loop until a letter "**q"** is entered at the keyboard. (Save the program as **whileq.c**)

```
#include<stdio.h>
#include<conio.h>
main()
    {
      char ch;
      ch = getche();
      while(ch != 'q')
          ch = getche();
          printf("\nFound the q");
    }
```

Example 5

This program prints the numbers 1 to 5 using a **do while** loop. (Save the program as: **1to5.c**)

```
   #include<stdio.h>
   main()
      {
           int I;
           I = 0;
         do
          {
            printf(" The value of I is now %d\n", I);
            I=I+1;
          } while (I < 6);
       }
```

Example 6

This program adds numbers entered by the user until the user enters zero. (Save the program as **addition.c**)

```
#include <stdio.h>
main()
    {
      double number, sum = 0;
      do   {
             printf("Enter a number: ");
             scanf("%lf", &number);
             sum += number;
             }while(number != 0.0);

      printf("Sum = %f",sum);

      }
```

Example 7

The **switch** statement is often used to process menu commands. This program will add, subtract, multiply or divide two numbers. (Save the program as **switch2.c**)

```
#include <stdio.h>
main()
  {
    int a, b;
    char ch;
    printf("***Menu***\n");
    printf("1: Add\n");
    printf("2: Subtract\n");
    printf("3: Multiply\n");
    printf("4: Divide\n");
do
   {
     printf("Enter A (for Add), S (for Subtract) ");
     printf("M (for Multiply) and D (for Divide)\n");
     printf("Enter your Choice: ");
     ch = getchar();
   }
    while (ch != 'A' && ch != 'S' && ch != 'M' && ch != 'D');
          printf("\n\n");
          printf("Enter first number:    ");
          scanf("%d", &a);
          printf("Enter second number:   ");
          scanf("%d", &b);

    switch (ch)
          {
          case 'A':
               printf("%d", a + b);
               break;
          case 'S':
               printf("%d", a - b);
               break;
          case 'M':
               printf ("%d", a * b);
               break;
          case 'D':
               if (b != 0)
               printf("%d", a/b);
               break;
          }
}
```

5.0 FUNCTIONS

A function is a block of statements that performs a specific task. Most real world programs contain many functions. Before we can use a function, we define a function prototype.

5.1 Function Prototype

Function prototype declares four attributes associated with a function

(i) The function name
(ii) Its return type
(iii) The number of its parameters
(iv) The type of its parameters

A **function prototype** consists of a functions name, its return types and its parameter lists. Examples of a function prototype are:

```
void myfunct(void);
int func(void);
void sum(int x, int y);
```

In the function prototype

```
void sum(int x, int y);
```

`void` is the return type, `sum` is a function name and `int x, int y` are parameters.

Function Prototypes provide several benefits:

(i) They inform the compiler about the return type of a function
(ii) They enable the compiler to report when the number of arguments passed to a function is not the same as the number of parameters declared by the function.

When you call a function, the compiler needs to know the type of data returned by that function. If you use a function that is not prototyped, then the compiler will simply assume that the returned value is an integer.

5.2 Function Definition

Generally a function definition has this form:

```
return-type function-name(parameter declaration, if any)
    {
     declarations
     statements
    }
```

The only function that does not need a prototype is `main()` since it is predefined by the C language. When a function is called, execution transfers to that function. When a function ends, execution resumes at the point in your program immediately following the call to the function. The value that a called function computes may be returned to the calling function by using the `return` statement

i.e. `return expression;`

A function need not return a value. A return statement with no expression causes control, but no useful value to be returned to the caller. Since main is a function like any other, it may return a value to its caller, which is in effect the environment in which the program was executed. A return value of zero implies normal termination. Non-zero values signal unusual or erroneous termination condition

The function prototype declaration

```
int power(int m, int n);
```

means that `power` is a function that expects two `int` arguments and returns an `int`.

This declaration has to agree with the definition and uses of `power`. Any function inside a program may call any other function within the same program. Traditionally, `main()` is not called by any other function. Remember a function call is a statement, so a semicolon must terminate it.

Example 1

This program uses two functions, `main()` and `funct1()`. (Save the program as **Hello.c**)

```
/* A program with two functions*/
#include <stdio.h>
void funct1(void);    /* prototype for funct1() */
main()
     {
       printf("Hello     ");
       funct1();                    /*calling funct1( ) */
       printf("This is a program involving two functions");
     }
void funct1(void)
     {
       printf("World. ")          /*called function*/
     }
```

Note: This program will display:

Hello world. This is a program involving two functions

Example 2

```
/* A program that returns a value */
# include <stdio.h>
int funct(void);      /*Function prototype*/
main( )
    {
       int num;
       num = funct();      /* calling funct()*/
       printf("The number is %d", num);
    }
int funct(void)            /* Called function */
   {
     return 10;
   }
```

Note: In this program, `funct()` returns an integer value to the calling `main()` function and 10 is assigned to num.

5.3 Function Arguments

Definition:

A functions argument is a value that is passed to the function when the function is called. A function in C can have from zero to several arguments. (The upper limit is determined by the compiler you are using, but the standard ANSI specifies that a function must be able to take at least 31 arguments).

Example 3

This program demonstrates the use of arguments and how to pass arguments to the called function. (Save the program as **argument.c**)

```
#include <stdio.h>
void sum(int x,int y);    /* The function sum() receives two integer values x and y */
main( )
   {
      sum(2, 20);      /* sending 2 and 20 to sum ( ) as values of x and y respectively */
      sum(40, 5);
   }
  void sum(int x, int y)
  {
      printf("%d\n", x + y);
   }
```

Note: When `sum()` is called, the value of each argument is copied into its marching parameter.

In the first call `sum(2,20)`, 2 is copied into x and 20 into y, in the second call `sum(40,5)`, 40 is copied to x and 5 is copied to y.

Example 4

A program that calculates the area of several circles .Save the program as **circles.c**

/* Program to calculate the area of circles, using the for loop */

```
#include<stdio.h>
float process(float radius);
main()
  {
     float radius, area;              /* variable declaration */
     int count, n;                    /* variable declaration */

     printf("How many circles?  ");
     scanf("%d", &n);
```

```
   for (count = 1; count <= n; ++count)
          {
             {
               printf("\nCircle no. %d: Radius =   ", count);
               scanf("%f", &radius);
             }
           if (radius < 0)
              area = 0;
           else
              area = process(radius);

      printf("Area = %f\n", area);
          }
  }
float process(float r)          /* function definition */
  {
      float a, pi;              /*local variable declaration */
      pi = 3.14159;
      a = pi * r * r;
      return(a);
  }
```

Exercise 6

Q.1 The moons gravity is about 17% of the Earths gravity. Write a program that allows you to enter your weight and computes your effective weight on the moon.
You may use only one function, i.e. `main()` or you may add another function dedicated for computation) (save the program as **gravity**)

Q.2 Write a program to convert temperatures from degrees Fahrenheit to degree centigrade [you may use the formula C = (5/9)(F – 32)]. Let the program prompts the user to input values of Fahrenheit, F from the key board. Use only one main functions main. (Save the program as **degree**)

Q.3 Modify question two (Q2) above using another function, other than main, to do the conversion. (Save the program as **degree_modified**)

Q4. Write a program with two functions; the first one is the `main()`. This will prompt the user to input radius of a sphere and the second one, `myvol()`, will calculate the volume of a sphere and return a value to `main()` that will print the result. (Save the program as **Sphere_volume**)

Q.5 Write a program that displays the square of a number entered from the keyboard.
The square number is computed using the function **squared( ).** (Save the program as **square**)

Q.6 Challenge problem.
The following program intends to use a function called `convert()`, which prompts the user for an amount in USD and convert the value into TSH, (Using an exchange rate of Tshs.2500/= per US dollar). It is then expected to return this value to main program so as

to display the conversion. Examine the code and explain why it can't give the expected result. Rewrite the program so that it can work as expected. (Save the program as **convert**)

```
#include<stdio.h>
float convert(void);
main()
 {
   float USD, TSH;
   TSH = convert();
   printf("USD %f = TSH %f",USD,TSH);
 }

float convert()
 {

 float dollar;
 printf("Enter amount in USD:  ");
 scanf("%f", &dollar);
 return dollar * 2500;
}
```

Re-write the code so that it can work properly

More examples

1. To see how a function prototype can catch an error, try to compile this program (save it as vol.cpp)

```
#include <stdio.h>
float volume(float S1, float S2, float S3);
main ()
   {
      float vol;
      vol = volume (20.5, 5.5, 10.5, 15.5);   /*error*/
      printf("Volume = %f", vol);
   }

 /*compute volume*/
 float volume(float S1 float S2, float S3)
   {
      return S1*S2*S3;
   }
```

Note: This program will not compile because the volume () function is declared as having only 3 parameters, but the program is attempting to call it with 4 parameters

2. Analyze the following program and state whether it is correct or not. If not state why?

```
#include<stdio.h>
foat myfunc( float num);
main()
    {
      printf("%f", myfunc(10.5);
    }
float myfunc(float num);
    {
      return num * num;
    }
```

Exercise 7

Q1. Write a program that uses a function called myvolume(), let the program compute the volume of a cylinder from the formula $\mathbf{v = \pi r^2 h}$. Have the main() function sends the values of **r** and **h** to myvolume() which perform the calculation and return the result to main to be displayed (Save the program as **cylinder**)

Q2. Write a program with two functions; the first one is the **main( )**. This will prompt the user to input radius of a sphere and the second one, **myvol( )**, will calculate the volume of a sphere and return a value to **main( )** that will print the result.
(The Volume of a sphere is given by: $\mathbf{4/3 * \pi * r^3}$) (Save the program as **SphereVolume**)

Q3. **Challenge problem**:
Write a program, which uses **convert.c)** and **Gravity** as functions. (Save the program as **combined**)

Exercise 8

Q. 1 Write a program that converts miles to kms using a '**do**' loop, allow the user to repeat the conversion (1 mile = 1.6093km). Save the program as **mile.**

Q. 2 Write a program that displays the menu below and uses a do loop to check for valid responses (your program does not need to implement the actual function shown in the menu. Save the program as **menu**

Q. 3 Write a program to calculate the area of a triangle, allowing only positive values to be entered for the base and height.

Q. 4 (Challenge Problem) Write a program that computes the area of either a circle, rectangle or triangle using the if-else-if ladder. (Save the program as **ifladder**)

Exercise 9

Q.1 Using switch–case statements write a program that compute the area of a circle, the area of a rectangle and the area of a triangle. Let the program prompt the user to enter dimensions, perform calculations and print the result on each case.

Q.2 Write a C program to read a "float" number representing temperatures. Let your program define two functions convert_Fuhr() and convert_Cels(), whereby convert_Fuhr() converts degree Fahrenheit to degree Celsius and convert_Cels() convert degree Celsius to Fahrenheit. Your main program should print the float equivalent temperature results such us:
"100.0 DEGREES CELSIUS EQUALS 212.0 DEGREE FAHRENHEIT".
After the user has entered a number, let him have a choice of what conversion he want to perform.

Q.3 Write a program that reads two "floating" point numbers representing the radius and height of a cylinder. Declare two functions that calculate the area and volume of a cylinder and let the program print out the area and volume for the given dimensions. Your output should take the form.

The area of a cylinder of radius …cm and height …cm is … square cm.
The volume of a cylinder of radius …cm and height …cm is … square cm.

Also let your program print the error message

"ERROR!! Negative values not permitted!!"

In case the user enters negative values for radius or height.

Q.4 Using the "switch" statement write a programme that recognizes the numbers 2, 4, 6, 8 and 10. Let the programme display the word TWO when the number two is entered, FOUR, when the number 4 is entered and so on. Let also the program display the statement UNRECOGNIZED INPUT!, when the number entered in not among the numbers mentioned above**.**

Exercise 10

1. When do you use the keyword **return** while defining a function? When do you not use the keyword **return** when defining a function?

2. Write a program in C that prints out the larger of two numbers entered from the keyboard. Use a function to do the actual comparison of the two numbers. Pass the two numbers to the function as arguments, and have the function return the answer.

3. Write a program in C that asks the user to input the length of two sides of the right-angled triangle. Make use of the built-in functions **pow()** and **sqrt()** to compute the length of hypotenuse using Pythagoras Theorem.

4. Write a C program that asks for a distance in meters and convert it to feet.

5. Write a C program that calls a function which returns a cube of a given number.

6. Write a programme to accept a decimal number and convert it to binary number.

ALGORITHM

In computer programming, an algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

Example 1

Write an algorithm to read two numbers and find their sum.

Algorithm

Step 1: Start
Step 2: Declare variables num1, num2 and sum
Step 3: Read values for num1 and num2
Step 4: sum←num1+num2
Step 5: Display sum
Step 6: End

Example 2

Write an algorithm to compute the area of a rectangle

Algorithm

Step 1: Start
Step 2: Declare variables length, width and area
Step 3: Read values for length and width
Step 4: area← length * width
Step 5: Display area
Step 6: Stop

Example 3

Write an algorithm to convert temperature Fahrenheit to Celsius

Algorithm

Step 1: Start
Step 2: Declare variables F and C.
Step 3: Read temperature in Fahrenheit: F
Step 4: C← 5/9 * (F - 32)
Step 5: Print Temperature in Celsius: C
Step 6: End

Exercise

1. Write an algorithm to print the square of a number entered by user

2. Write an algorithm to calculate volume of the cylinder

3. Write an algorithm to print all numbers between 1 and 100 inclusively